FACULDADE

Banco de Dados I

DOCUMENTAÇÃO DO BANCO DE DADOS

Sistema MEI — Gestão Financeira para Microempreendedores Individuais

Docente: Fernanda Regebe

Discente: Gabriel Conrado da Silva

Camaçari

2026

DOCUMENTAÇÃO DO BANCO DE DADOS

Sistema MEI — Gestão Financeira para Microempreendedores Individuais

Documentação técnica do banco de dados relacional do Sistema MEI, desenvolvido como trabalho prático da disciplina Banco de Dados I, contendo estrutura de tabelas, relacionamentos, índices, inserção de dados e consultas SQL com JOIN.

Docente: Fernanda Regebe

Camaçari

2026

SUMÁRIO

1 APRESENTAÇÃO……1

2 VISÃO GERAL DO BANCO DE DADOS…….2

3 ESTRUTURA DAS TABELAS……3

3.1 Tabela CATEGORIA…….4

3.2 Tabela METODO_PAGAMENTO……5

3.3 Tabela USUARIO……..6

3.4 Tabela VENDAS……8

3.5 Tabela NOTA_FISCAL…….10

3.6 Tabela PAGAMENTO……...12

4 RELACIONAMENTOS E CHAVES ESTRANGEIRAS……..14

4.1 Cardinalidades……15

4.2 Definição SQL das foreign keys……15

5 ÍNDICES…….16

6 DADOS DE CONTEXTO…….17

6.1 Categorias……...17

6.2 Métodos de pagamento……...18

6.3 Usuários……..18

6.4 Vendas…….18

6.5 Notas fiscais……19

6.6 Pagamentos…….19

7 CONSULTAS SELECT COM JOIN20

7.1 Consulta 1 — Vendas completas com categoria e usuário……..21

7.2 Consulta 2 — Notas fiscais completas……..……..……22

7.3 Consulta 3 — Análise de pagamentos por método…….23

7.4 Consulta 4 — Vendas por mês e categoria……..……..……..24

7.5 Consulta 5 — Top 5 categorias mais lucrativas……...25

8 ESTRATÉGIA DE SOFT DELETE……..……26

1 APRESENTAÇÃO

Este documento apresenta a documentação técnica completa do banco de dados relacional utilizado no Sistema MEI — aplicação web desenvolvida em Java com Jakarta EE para auxiliar Microempreendedores Individuais no controle financeiro de suas atividades e no cumprimento das obrigações fiscais previstas em lei.

O banco de dados, denominado mei, foi modelado no SGBD MySQL/MariaDB 10.4+ e é composto por seis tabelas inter-relacionadas que suportam o ciclo completo de operação do sistema: cadastro de usuários MEI, categorização e registro de vendas, emissão de notas fiscais e controle de formas de pagamento.

Esta documentação cobre todas as camadas de definição do banco: criação das tabelas (CREATE TABLE), relacionamentos por chaves estrangeiras (ALTER TABLE / FOREIGN KEY), estratégia de soft delete, índices de performance, inserção dos dados de contexto (INSERT INTO) e as cinco consultas SQL com JOIN exigidas como requisito acadêmico do trabalho.

2 VISÃO GERAL DO BANCO DE DADOS

O banco mei possui as seguintes características gerais de configuração:

- SGBD: MySQL 8.0 / MariaDB 10.4+;
- Engine: InnoDB — com suporte a transações ACID e chaves estrangeiras;
- Charset: utf8mb4 com collation utf8mb4_unicode_ci;
- Quantidade de tabelas: 6;
- Quantidade de foreign keys: 6;
- Índices criados: 15;
- Soft delete implementado em todas as tabelas principais;
- Timestamps automáticos via DEFAULT current_timestamp().

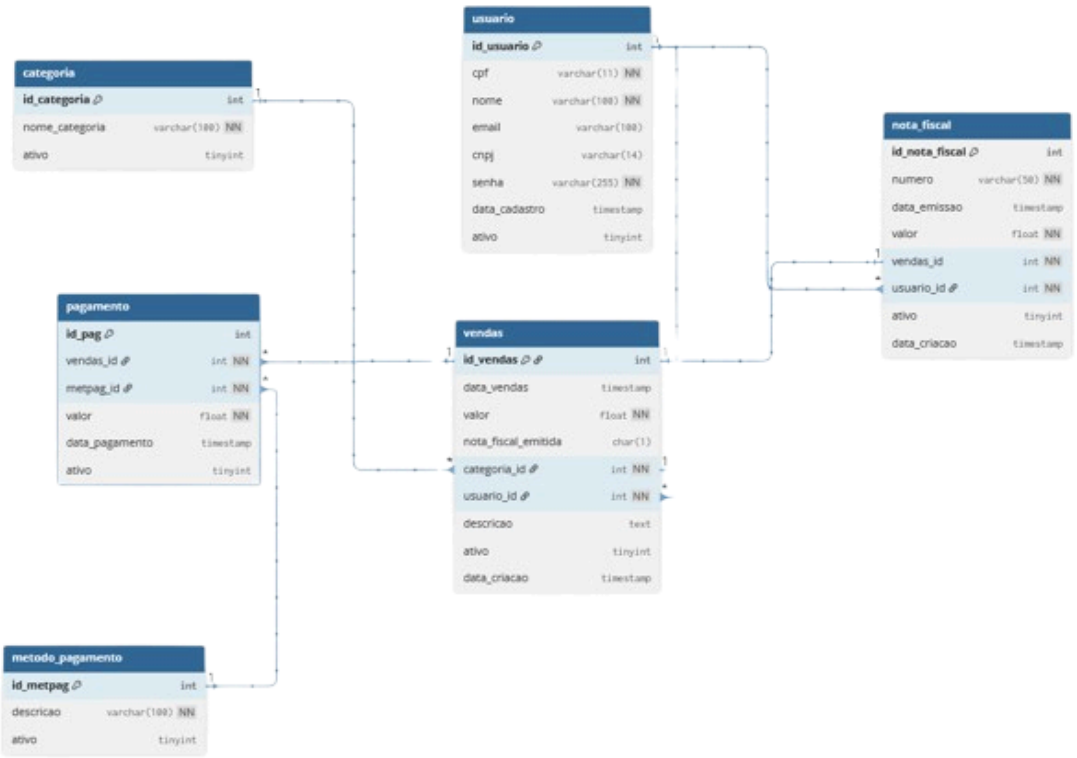
Todas as tabelas adotam a convenção snake_case para nomes. As chaves primárias seguem o padrão id_[tabela] (ex.: id_usuario) e as chaves estrangeiras seguem o padrão

[tabela]_id (ex.: usuario_id). O campo ativo TINYINT(1) DEFAULT 1 está presente em todas as tabelas principais para viabilizar o soft delete — exclusão lógica sem remoção física do registro.

3 ESTRUTURA DAS TABELAS

O banco de dados possui seis tabelas que representam as entidades do domínio. A Figura 1 apresenta o diagrama entidade-relacionamento completo do banco, seguido da descrição detalhada de cada tabela com seus campos, tipos, restrições e definição SQL.

Figura 1 — Diagrama entidade-relacionamento do banco mei



3.1 Tabela CATEGORIA

Armazena as categorias de classificação das vendas realizadas pelo MEI. Permite organizar o faturamento por segmento de atividade, viabilizando relatórios e análises por tipo de receita.

Tabela 1 — Estrutura da tabela categoria

Campo	Tipo	Nulo	Chave	Descrição
id_categoria	INT(11)	NÃO	PK	Identificador único — chave primária, auto increment
nome_categoria	VARCHAR(100)	NÃO	—	Nome descritivo da categoria
ativo	TINYINT(1)	SIM	—	Soft delete: 1 = ativa, 0 = inativa

Fonte: elaborado pelos autores (2026).

Definição SQL:

```
CREATE TABLE `categoria` (
  `id_categoria`   int(11)      NOT NULL,
  `nome_categoria` varchar(100) NOT NULL,
  `ativo`          tinyint(1)   DEFAULT 1
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4
    COLLATE=utf8mb4_unicode_ci;
```

3.2 Tabela METODO_PAGAMENTO

Tabela de domínio que armazena os métodos de pagamento disponíveis no sistema. É referenciada pela tabela pagamento, permitindo identificar de que forma cada venda foi liquidada.

Tabela 2 — Estrutura da tabela metodo_pagamento

Campo	Tipo	Nulo	Chave	Descrição
id_metpag	INT(11)	NÃO	PK	Identificador único — chave primária, auto increment
descricao	VARCHAR(100)	NÃO	—	Nome do método (Dinheiro, PIX, Cartão, etc.)
ativo	TINYINT(1)	SIM	—	Soft delete: 1 = ativo, 0 = inativo

Fonte: elaborado pelos autores (2026).

```
CREATE TABLE `metodo_pagamento` (
  `id_metpag`  int(11)      NOT NULL,
  `descricao`  varchar(100) NOT NULL,
  `ativo`      tinyint(1)   DEFAULT 1
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4
    COLLATE=utf8mb4_unicode_ci;
```

3.3 Tabela USUARIO

Armazena os dados dos Microempreendedores Individuais cadastrados no sistema. O CPF é utilizado como identificador de login e a senha é armazenada em hash BCrypt com salt aleatório, garantindo segurança mesmo em caso de comprometimento do banco.

Tabela 3 — Estrutura da tabela usuario

Campo	Tipo	Nulo	Chave	Descrição
id_usuario	INT(11)	NÃO	PK	Identificador único — chave primária, auto increment
cpf	VARCHAR(11)	NÃO	UNIQUE	CPF do MEI (11 dígitos, sem máscara) — login
nome	VARCHAR(100)	NÃO	—	Nome completo do MEI
email	VARCHAR(100)	NÃO	UNIQUE	E-mail único — recuperação de senha
cnpj	VARCHAR(14)	SIM	—	CNPJ do MEI (14 dígitos) — opcional
senha	VARCHAR(255)	NÃO	—	Hash BCrypt da senha (60 caracteres)
data_cadastro	TIMESTAMP	NÃO	—	Data/hora do cadastro — automático
ativo	TINYINT(1)	SIM	—	Soft delete: 1 = ativo, 0 = inativo

Fonte: elaborado pelos autores (2026).

```
CREATE TABLE `usuario` (
  `id_usuario`    int(11)      NOT NULL,
  `cpf`           varchar(11)  NOT NULL,
  `nome`          varchar(100) NOT NULL,
  `email`         varchar(100) NOT NULL,
  `cnpj`          varchar(14)  DEFAULT NULL,
  `senha`         varchar(255) NOT NULL,
  `data_cadastro` timestamp    NOT NULL DEFAULT current_timestamp(),
  `ativo`         tinyint(1)   DEFAULT 1
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4
    COLLATE=utf8mb4_unicode_ci;
```

3.4 Tabela VENDAS

Tabela central do sistema. Registra cada venda realizada pelo MEI, constituindo o histórico financeiro que alimenta os relatórios mensais, o dashboard e as análises de faturamento. Possui relacionamentos com usuario, categoria, nota_fiscal e pagamento.

Tabela 4 — Estrutura da tabela vendas

Campo	Tipo	Nulo	Chave	Descrição
id_vendas	INT(11)	NÃO	PK	Identificador único — chave primária, auto increment

Campo	Tipo	Nulo	Chave	Descrição
data_vendas	TIMESTAMP	NÃO	—	Data/hora da venda — automático
valor	FLOAT	NÃO	—	Valor total da venda em Reais
nota_fiscal_emitida	CHAR(1)	SIM	—	NF emitida: 'S' = Sim, 'N' = Não (CHECK constraint)
categoria_id	INT(11)	NÃO	FK	Referência à tabela categoria (N:1)
usuario_id	INT(11)	NÃO	FK	Referência à tabela usuario (N:1)
descricao	TEXT	SIM	—	Descrição opcional da venda (texto livre)
ativo	TINYINT(1)	SIM	—	Soft delete: 1 = ativa, 0 = inativa
data_criacao	TIMESTAMP	NÃO	—	Timestamp de criação do registro

Fonte: elaborado pelos autores (2026).

```
CREATE TABLE `vendas` (
  `id_vendas`           int(11)    NOT NULL,
  `data_vendas`         timestamp  NOT NULL DEFAULT
current_timestamp(),
  `valor`               float      NOT NULL,
  `nota_fiscal_emitida` char(1)    DEFAULT 'N'
    CHECK (`nota_fiscal_emitida` in ('S','N')),
  `categoria_id`        int(11)    NOT NULL,
  `usuario_id`          int(11)    NOT NULL,
  `descricao`           text       DEFAULT NULL,
  `ativo`               tinyint(1) DEFAULT 1,
  `data_criacao`        timestamp  NOT NULL DEFAULT current_timestamp()
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4
    COLLATE=utf8mb4_unicode_ci;
```

3.5 Tabela NOTA_FISCAL

Armazena os dados das notas fiscais emitidas pelo MEI. Possui relacionamento 1:1 com a tabela vendas — o campo vendas_id é UNIQUE, garantindo que cada venda tenha no máximo uma nota fiscal. Registros de NF não são removidos fisicamente para manter conformidade fiscal e rastreabilidade de auditoria.

Tabela 5 — Estrutura da tabela nota_fiscal

Campo	Tipo	Nulo	Chave	Descrição
id_nota_fiscal	INT(11)	NÃO	PK	Identificador único — chave primária, auto increment
numero	VARCHAR(50)	NÃO	UNIQUE	Número da nota fiscal (único na base)
data_emissao	TIMESTAMP	NÃO	—	Data/hora de emissão — automático
valor	FLOAT	NÃO	—	Valor da NF (deve coincidir com vendas.valor)
vendas_id	INT(11)	NÃO	FK/UQ	Referência à venda (1:1 — campo UNIQUE)

Campo	Tipo	Nulo	Chave	Descrição
usuario_id	INT(11)	NÃO	FK	Referência ao usuário emissor (N:1)
ativo	TINYINT(1)	SIM	—	Soft delete: 1 = ativa, 0 = cancelada
data_criacao	TIMESTAMP	NÃO	—	Timestamp de criação do registro

Fonte: elaborado pelos autores (2026).

```
CREATE TABLE `nota_fiscal` (
  `id_nota_fiscal` int(11)     NOT NULL,
  `numero`         varchar(50) NOT NULL,
  `data_emissao`   timestamp   NOT NULL DEFAULT current_timestamp(),
  `valor`          float       NOT NULL,
  `vendas_id`      int(11)     NOT NULL,
  `usuario_id`     int(11)     NOT NULL,
  `ativo`          tinyint(1)  DEFAULT 1,
  `data_criacao`   timestamp   NOT NULL DEFAULT current_timestamp()
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4
    COLLATE=utf8mb4_unicode_ci;
```

3.6 Tabela PAGAMENTO

Registra de que forma cada venda foi paga. Permite múltiplos registros por venda, suportando pagamento misto (ex.: R$ 50,00 em dinheiro + R$ 100,00 no cartão) e pagamento parcelado (ex.: 3x no cartão de crédito). A validação de que a soma dos pagamentos corresponde ao valor total da venda é realizada na camada Java, não por trigger no banco.

Tabela 6 — Estrutura da tabela pagamento

Campo	Tipo	Nulo	Chave	Descrição
id_pag	INT(11)	NÃO	PK	Identificador único — chave primária, auto increment
vendas_id	INT(11)	NÃO	FK	Referência à venda (N:1)
metpag_id	INT(11)	NÃO	FK	Referência ao método de pagamento (N:1)
valor	FLOAT	NÃO	—	Valor deste pagamento em Reais
data_pagamento	TIMESTAMP	NÃO	—	Data/hora do pagamento — automático
ativo	TINYINT(1)	SIM	—	Soft delete: 1 = ativo, 0 = inativo

Fonte: elaborado pelos autores (2026).

```
CREATE TABLE `pagamento` (
  `id_pag`         int(11)    NOT NULL,
  `vendas_id`      int(11)    NOT NULL,
  `metpag_id`      int(11)    NOT NULL,
  `valor`          float      NOT NULL,
  `data_pagamento` timestamp  NOT NULL DEFAULT current_timestamp(),
  `ativo`          tinyint(1) DEFAULT 1
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4
    COLLATE=utf8mb4_unicode_ci;
```

4 RELACIONAMENTOS E CHAVES ESTRANGEIRAS

O banco possui seis relacionamentos definidos por chaves estrangeiras, aplicados via ALTER TABLE após a criação das tabelas. Os relacionamentos garantem integridade referencial: não é possível inserir uma venda com categoria ou usuário inexistente, nem criar uma nota fiscal para uma venda inválida.

A estratégia de ON DELETE CASCADE foi aplicada nos relacionamentos em que a exclusão do registro pai deve propagar-se automaticamente para os dependentes. O comportamento RESTRICT padrão foi mantido nos casos em que a exclusão do pai seria um erro de negócio.

Tabela 7 — Relacionamentos (foreign keys) do banco mei

Constraint	Tabela origem	Tabela destino	Colunas	Comportamento
fk_vendas_categoria	vendas	categoria	categoria_id → id_categoria	RESTRICT
fk_vendas_usuario	vendas	usuario	usuario_id → id_usuario	ON DELETE CASCADE
fk_nota_fiscal_vendas	nota_fiscal	vendas	vendas_id → id_vendas	ON DELETE CASCADE
fk_nota_fiscal_usuario	nota_fiscal	usuario	usuario_id → id_usuario	ON DELETE CASCADE
fk_pagamento_vendas	pagamento	vendas	vendas_id → id_vendas	ON DELETE CASCADE
fk_pagamento_metpag	pagamento	metodo_pagamento	metpag_id → id_metpag	RESTRICT

Fonte: elaborado pelos autores (2026).

4.1 Cardinalidades

As cardinalidades entre as entidades do banco são as seguintes:

- USUARIO (1) — (N) VENDAS: um usuário pode ter muitas vendas; cada venda pertence a exatamente um usuário;
- USUARIO (1) — (N) NOTA_FISCAL: um usuário pode ter muitas notas fiscais; cada NF pertence a um usuário;

— CATEGORIA (1) — (N) VENDAS: uma categoria classifica muitas vendas; cada venda possui exatamente uma categoria;

— VENDAS (1) — (1) NOTA_FISCAL: uma venda pode ter no máximo uma NF (campo vendas_id é UNIQUE em nota_fiscal);

— VENDAS (1) — (N) PAGAMENTO: uma venda pode ser paga com múltiplos registros de pagamento;

— METODO_PAGAMENTO (1) — (N) PAGAMENTO: um método pode aparecer em muitos pagamentos.

4.2 Definição SQL das foreign keys

As seis restrições de integridade referencial são definidas conforme o código a seguir:

```
-- FK: nota_fiscal -> usuario e vendas
ALTER TABLE `nota_fiscal`
  ADD CONSTRAINT `fk_nota_fiscal_usuario`
    FOREIGN KEY (`usuario_id`) REFERENCES `usuario` (`id_usuario`)
    ON DELETE CASCADE,
  ADD CONSTRAINT `fk_nota_fiscal_vendas`
    FOREIGN KEY (`vendas_id`) REFERENCES `vendas` (`id_vendas`)
    ON DELETE CASCADE;


-- FK: pagamento -> vendas e metodo_pagamento
ALTER TABLE `pagamento`
  ADD CONSTRAINT `fk_pagamento_vendas`
    FOREIGN KEY (`vendas_id`) REFERENCES `vendas` (`id_vendas`)
    ON DELETE CASCADE,
  ADD CONSTRAINT `fk_pagamento_metpag`
    FOREIGN KEY (`metpag_id`) REFERENCES `metodo_pagamento`
(`id_metpag`);


-- FK: vendas -> categoria e usuario
ALTER TABLE `vendas`
  ADD CONSTRAINT `fk_vendas_categoria`
    FOREIGN KEY (`categoria_id`) REFERENCES `categoria`
(`id_categoria`),
  ADD CONSTRAINT `fk_vendas_usuario`
    FOREIGN KEY (`usuario_id`) REFERENCES `usuario` (`id_usuario`)
    ON DELETE CASCADE;
```

5 ÍNDICES

Foram criados 15 índices para garantir performance nas operações de JOIN, filtros por período e buscas por CPF e e-mail. Os índices são definidos via ALTER TABLE após a criação das tabelas e das chaves primárias.

Tabela 8 — Índices do banco mei e suas finalidades

Tabela	Índice	Finalidade
categoria	PRIMARY KEY (id_categoria)	Chave primária
categoria	INDEX (nome_categoria)	Busca por nome
metodo_pagamento	PRIMARY KEY (id_metpag)	Chave primária
usuario	PRIMARY KEY (id_usuario)	Chave primária
usuario	UNIQUE KEY (cpf)	CPF único — login
usuario	INDEX idx_email (email)	Busca por e-mail
vendas	PRIMARY KEY (id_vendas)	Chave primária
vendas	INDEX idx_usuario (usuario_id)	Filtro por MEI
vendas	INDEX idx_data (data_vendas)	Filtro por período
vendas	INDEX idx_categoria (categoria_id)	Filtro por categoria
nota_fiscal	PRIMARY KEY (id_nota_fiscal)	Chave primária
nota_fiscal	INDEX idx_numero (numero)	Busca por número da NF
nota_fiscal	INDEX idx_vendas (vendas_id)	JOIN com vendas
pagamento	PRIMARY KEY (id_pag)	Chave primária
pagamento	INDEX idx_data (data_pagamento)	Filtro por data

Fonte: elaborado pelos autores (2026).

6 DADOS DE CONTEXTO

Os dados inseridos no banco foram elaborados para demonstrar o funcionamento completo do sistema em ambiente de desenvolvimento. Todos os registros são de caráter exclusivamente ilustrativo.

6.1 Categorias

Sete registros inseridos. Os IDs 1 a 3 representam as categorias ativas padrão do sistema. Os IDs 4 a 6 são duplicatas com ativo = 0, inseridas para demonstrar o mecanismo de soft delete. O ID 7 é a categoria genérica "Outro".

```
INSERT INTO `categoria` VALUES
```

```
  (1, 'Revenda de Mercadorias',    1),
  (2, 'Produtos Industrializados', 1),
  (3, 'Prestacao de Servicos',     1),
  (4, 'Revenda de Mercadorias',    0), -- soft delete
  (5, 'Produtos Industrializados', 0), -- soft delete
  (6, 'Prestacao de Servicos',     0), -- soft delete
  (7, 'Outro',                     1);
```

6.2 Métodos de pagamento

Seis métodos ativos: Dinheiro, Cartão de Crédito, Cartão de Débito, PIX, Transferência Bancária e Boleto.

```
INSERT INTO `metodo_pagamento` VALUES
  (1, 'Dinheiro',               1),
  (2, 'Cartao de Credito',      1),
  (3, 'Cartao de Debito',       1),
  (4, 'PIX',                    1),
  (5, 'Transferencia Bancaria', 1),
  (6, 'Boleto',                 1);
```

6.3 Usuários

Seis usuários de teste inseridos. Todos possuem a senha "123456" armazenada como hash BCrypt. O campo CNPJ é NULL para a maioria, demonstrando que o campo é opcional. Um usuário (ID 6) possui CNPJ preenchido.

6.4 Vendas

Doze vendas inseridas, todas vinculadas ao usuário de ID 4. O conjunto contempla os seguintes cenários:

— vendas ativas sem nota fiscal (IDs 1, 2, 3, 5, 7, 8);

— vendas inativas por soft delete (IDs 4, 6, 10) — simulam exclusão lógica;

— vendas com nota fiscal emitida (IDs 9, 11, 12) — campo nota_fiscal_emitida = 'S';

— venda com valor elevado desativada (ID 4: R$ 243.234,00) — útil para testar exclusão de outliers.

6.5 Notas fiscais

Quatro notas fiscais inseridas, vinculadas às vendas 9, 10, 11 e 12. A nota vinculada à venda 10 está inativa, demonstrando o soft delete em nota fiscal.

6.6 Pagamentos

A tabela pagamento foi mantida vazia nos dados de contexto entregues. Para demonstração plena da Consulta 3, recomenda-se inserir ao menos um registro por método, conforme o exemplo abaixo:

```
INSERT INTO pagamento (vendas_id, metpag_id, valor) VALUES
  (1, 4, 345.00),  -- Venda 1 paga via PIX
  (2, 1,  34.00),  -- Venda 2 paga em Dinheiro
  (3, 2, 555.66);  -- Venda 3 paga no Cartao de Credito
```

7 CONSULTAS SELECT COM JOIN

O banco inclui cinco consultas SQL que utilizam JOIN entre múltiplas tabelas, atendendo ao requisito acadêmico do trabalho. Cada consulta tem aplicação prática direta no sistema — alimentando páginas JSP, relatórios PDF e o dashboard do MEI.

Tabela 9 — Resumo das cinco consultas SQL com JOIN

#	Consulta	Tabelas	Descrição
1	**Vendas com categoria e usuário**	vendas, categoria, usuario	Lista histórico completo com nome da categoria, vendedor e status de NF. INNER JOIN triplo, ORDER BY data DESC, LIMIT 20.
2	**Notas fiscais completas**	nota_fiscal, vendas, usuario	Relatório fiscal com número da NF, valor, descrição da venda e dados do emissor. Usa IFNULL para CNPJ ausente.
3	**Pagamentos por método**	vendas, pagamento, metodo_pagamento	Cruza venda com forma de pagamento. Identifica os métodos mais utilizados pelo MEI.
4	**Vendas por mês e categoria**	vendas, categoria	Agrupa por YEAR/MONTH e categoria com COUNT, SUM e AVG. Relatório anual de sazonalidade.
5	**Top 5 categorias lucrativas**	categoria, vendas	Ranking por receita total com percentual do faturamento global via subconsulta. GROUP BY + ORDER BY DESC + LIMIT 5.

Fonte: elaborado pelos autores (2026).

7.1 Consulta 1 — Vendas completas com categoria e usuário

Lista as últimas 20 vendas ativas com nome da categoria e do usuário. Utiliza INNER JOIN triplo entre vendas, categoria e usuario. Aplicação: historico.jsp e dashboard.jsp.

```
SELECT
    v.id_vendas                                        AS 'ID',
    DATE_FORMAT(v.data_vendas, '%d/%m/%Y %H:%i')       AS 'Data/Hora',
    CONCAT('R$ ', FORMAT(v.valor, 2, 'pt_BR'))         AS 'Valor',
    c.nome_categoria                                   AS 'Categoria',
    u.nome                                             AS 'Vendedor',
    v.nota_fiscal_emitida                              AS 'NF'
FROM vendas v
INNER JOIN categoria c ON v.categoria_id = c.id_categoria
INNER JOIN usuario   u ON v.usuario_id   = u.id_usuario
WHERE v.ativo = 1
ORDER BY v.data_vendas DESC
LIMIT 20;
```

7.2 Consulta 2 — Notas fiscais completas

Lista todas as notas fiscais ativas com detalhes da venda relacionada e dados do emissor. Usa IFNULL para exibir texto amigável quando o CNPJ não está cadastrado. Aplicação: RelatorioController para geração de PDF.

```
SELECT
    nf.numero                                          AS 'Numero NF',
    DATE_FORMAT(nf.data_emissao, '%d/%m/%Y %H:%i')    AS 'Data Emissao',
    CONCAT('R$ ', FORMAT(nf.valor, 2, 'pt_BR'))       AS 'Valor',
    v.descricao                                        AS 'Descricao da
Venda',
    u.nome                                             AS 'Emitido por',
    IFNULL(u.cnpj, 'Nao cadastrado')                  AS 'CNPJ do
Emissor'
FROM nota_fiscal nf
INNER JOIN vendas  v ON nf.vendas_id  = v.id_vendas
INNER JOIN usuario u ON nf.usuario_id = u.id_usuario
WHERE nf.ativo = 1
ORDER BY nf.data_emissao DESC;
```

7.3 Consulta 3 — Análise de pagamentos por método

Cruza vendas com pagamentos e métodos, permitindo identificar quais formas de recebimento são mais utilizadas. Não retorna resultados enquanto a tabela pagamento estiver vazia. Aplicação: card 'Formas de Pagamento' no dashboard.

```
SELECT
    v.id_vendas                                        AS 'ID Venda',
    DATE_FORMAT(v.data_vendas, '%d/%m/%Y')             AS 'Data Venda',
    CONCAT('R$ ', FORMAT(v.valor, 2, 'pt_BR'))         AS 'Total Venda',
    mp.descricao                                       AS 'Forma de
Pagamento',
    CONCAT('R$ ', FORMAT(p.valor, 2, 'pt_BR'))         AS 'Valor Pago'
FROM vendas v
INNER JOIN pagamento        p  ON v.id_vendas = p.vendas_id
INNER JOIN metodo_pagamento mp ON p.metpag_id = mp.id_metpag
WHERE v.ativo = 1 AND p.ativo = 1
ORDER BY p.data_pagamento DESC;
```

7.4 Consulta 4 — Vendas por mês e categoria

Relatório anual que agrupa vendas por mês, ano e categoria. Usa COUNT, SUM e AVG para calcular quantidade de vendas, total faturado e ticket médio por segmento. Aplicação: relatorio.jsp — análise de sazonalidade.

```
SELECT
    MONTH(v.data_vendas)                               AS 'Mes',
    YEAR(v.data_vendas)                                AS 'Ano',
    c.nome_categoria                                   AS 'Categoria',
    COUNT(*)                                           AS 'Qtd Vendas',
    CONCAT('R$ ', FORMAT(SUM(v.valor), 2, 'pt_BR'))  AS 'Total
Faturado',
    CONCAT('R$ ', FORMAT(AVG(v.valor), 2, 'pt_BR'))  AS 'Ticket Medio'
FROM vendas v
INNER JOIN categoria c ON v.categoria_id = c.id_categoria
WHERE v.ativo = 1 AND YEAR(v.data_vendas) = 2026
GROUP BY YEAR(v.data_vendas), MONTH(v.data_vendas),
         c.id_categoria, c.nome_categoria
ORDER BY YEAR(v.data_vendas) DESC,
         MONTH(v.data_vendas) DESC, SUM(v.valor) DESC;
```

7.5 Consulta 5 — Top 5 categorias mais lucrativas

Ranking das cinco categorias com maior receita total. Inclui subconsulta para calcular o percentual de cada categoria sobre o faturamento global. Usa GROUP BY, ORDER BY DESC e LIMIT 5. Aplicação: card estratégico no dashboard.

```
SELECT
    c.nome_categoria                                        AS
'Categoria',
    COUNT(v.id_vendas)                                      AS 'Qtd
Vendas',
    CONCAT('R$ ', FORMAT(SUM(v.valor), 2, 'pt_BR'))     AS 'Total
Faturado',
    CONCAT('R$ ', FORMAT(AVG(v.valor), 2, 'pt_BR'))     AS 'Ticket
Medio',
    CONCAT(
        ROUND((SUM(v.valor) /
          (SELECT SUM(valor) FROM vendas WHERE ativo=1)
        ) * 100, 1), '%'
    )                                                       AS
'Percentual'
FROM categoria c
INNER JOIN vendas v ON v.categoria_id = c.id_categoria
WHERE v.ativo = 1 AND c.ativo = 1
GROUP BY c.id_categoria, c.nome_categoria
ORDER BY SUM(v.valor) DESC
LIMIT 5;
```

8 ESTRATÉGIA DE SOFT DELETE

Todas as tabelas principais do banco adotam soft delete — exclusão lógica em vez de remoção física dos registros. O mecanismo é implementado pelo campo ativo TINYINT(1) DEFAULT 1, presente nas seis tabelas.

Quando um registro precisa ser desativado, a aplicação Java executa um UPDATE definindo ativo = 0, e não um DELETE. Isso garante:

- histórico financeiro preservado mesmo após desativação de vendas ou usuários;
- conformidade fiscal: notas fiscais canceladas permanecem no banco para auditoria;
- integridade referencial mantida sem disparar os CASCADEs das foreign keys;
- possibilidade de reativação de registros desativados por engano.

Todas as consultas SELECT do sistema incluem o filtro WHERE ativo = 1 para garantir que registros desativados não aparecem nos relatórios e telas do usuário.

```
-- Soft delete (uso correto)
UPDATE vendas SET ativo = 0 WHERE id_vendas = 10;
```

```
-- DELETE fisico (proibido em producao)
-- DELETE FROM vendas WHERE id_vendas = 10;
```